

JAVA PROGRAMMING LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. MANISH SIR

SUBMMITED BY:

HASEEN BABU

INDEX / TABLE OF CONTENTS

Applied Data Structure & Algorithms Lab (BCS553)

S.N.	PROGRAM NAME / EXPERIMENT TITLE	PAGE	DATE	SIGN
1	Reading Array Values	1	01/08/2026	
2	Largest, Smallest and Count	2	08/08/2026	
3	Bubble Sort	3	15/08/2026	
4	Selection Sort	4	22/08/2026	
5	Linear Search	5	29/08/2026	
6	Binary Search on a Sorted Array	6	05/09/2026	
7	Practice Set (Array, Loops, Search)	7	12/09/2026	
8	Classes and Objects	8	19/09/2026	
9	Constructors and Field Initialization	9	26/09/2026	
10	Parent and Child Classes (Inheritance)	10	03/10/2026	
11	Method Overriding & Polymorphism	11	10/10/2026	
12	Recursive Factorial, Sum and Power	12	17/10/2026	
13	Recursion on Arrays	13	24/10/2026	
14	Sorted Check, Search and Fibonacci	14	31/10/2026	
15	Subsets and Arrangements (Backtracking)	15	07/11/2026	

JAVA - EXPERIMENT NO. 01
Reading Array Values

1. EXPERIMENT TITLE

Reading Array Values using Indexes and Loops

2. AIM

Read values from an array by index and by a loop.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, Eclipse / IntelliJ IDEA / VS Code, Command Prompt / Terminal

4. THEORY

An array is a linear data structure holding homogeneous elements in contiguous memory. Array indices are zero-based (`0` to `n-1`). Loops allow sequential access for aggregation and traversal.

5. ALGORITHM / PROCEDURE

Task 1: Declare array, access `[0]` and `[length-1]`, print values.
Task 2: Iterate array from `0` to `length-1` using `for` loop, printing each element.
Task 3: Maintain `sum` variable, accumulate array elements inside loop, display sum.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Read by Index

```
public class Task1 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        System.out.println("First: " + marks[0]);
        System.out.println("Last: " + marks[marks.length - 1]);
    }
}
```

Task 2: Print using Loop

```
public class Task2 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

Task 3: Array Sum

```
public class Task3 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int sum = 0;
        for (int m : marks) {
            sum += m;
        }
        System.out.println("Sum: " + sum);
    }
}
```

7. INPUT

Array: `int[] marks = {45, 67, 80, 52, 90}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 01

```
c:\java> java Task1 First: 45 Last: 90 c:\java> java Task2 45 67 80 52 90 c:\java>
java Task3 Sum: 334
```

9. RESULT

The experiment was performed successfully. Array indexing and loop traversal were implemented correctly.

10. VIVA-VOCE QUESTIONS

Q1: What is zero-based indexing?
A: Array indices start at 0, so the first element is at index 0 and the last is at index length-1.

Q2: What exception occurs on illegal array access?
A: `ArrayIndexOutOfBoundsException`.

JAVA - EXPERIMENT NO. 02
Largest, Smallest and Count

1. EXPERIMENT TITLE

Finding Largest/Smallest Values and Counting Elements in Array

2. AIM

Find the largest and smallest value in an array and count values that pass a test.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse / IntelliJ IDEA

4. THEORY

Extreme value searches evaluate elements sequentially against a tracked extrema variable. Conditional filtering checks each item against a predicate (e.g. `> 60`) to accumulate matching tallies.

5. ALGORITHM / PROCEDURE

Task 1: Set `largest = marks[0]`. Loop through array; if `marks[i] > largest`, update `largest`.
Task 2: Set `smallest = marks[0]`. Loop through array; if `marks[i] < smallest`, update `smallest`.
Task 3: Set `count = 0`. Loop through array; if element `> 60`, increment `count`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Find Largest

```
public class Task1 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int largest = marks[0];
        for (int i = 1; i < marks.length; i++) {
            if (marks[i] > largest) largest = marks[i];
        }
        System.out.println("Largest: " + largest);
    }
}
```

Task 2: Find Smallest

```
public class Task2 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int smallest = marks[0];
        for (int i = 1; i < marks.length; i++) {
            if (marks[i] < smallest) smallest = marks[i];
        }
        System.out.println("Smallest: " + smallest);
    }
}
```

Task 3: Count Values > 60

```java
public class Task3 {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int count = 0;
        for (int m : marks) {
            if (m > 60) count++;
        }
        System.out.println("Count: " + count);
    }
}
```

7. INPUT

Array: `int[] marks = {45, 67, 80, 52, 90}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 02

```
c:\java> java Task1 Largest: 90 c:\java> java Task2 Smallest: 45 c:\java> java Task3
Count: 3
```

9. RESULT

The experiment was completed successfully. Max, min, and count calculations executed as expected.

10. VIVA-VOCE QUESTIONS

Q1: Why initialize `largest` with `marks[0]` instead of 0?
A: Initializing with 0 fails if all elements in the array are negative numbers.

Q2: What is the time complexity of finding the max element?
A: O(n) because each element is visited once.

JAVA - EXPERIMENT NO. 03
Bubble Sort

1. EXPERIMENT TITLE

Implementation of Bubble Sort Algorithm

2. AIM

Sort an array by comparing neighbouring values and swapping them when they are out of order.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, Terminal / IDE

4. THEORY

Bubble sort continuously compares adjacent elements and swaps them if unsorted. In each pass, the largest remaining element bubbles up to its correct rightmost position. Time complexity is $O(n^2)$.

5. ALGORITHM / PROCEDURE

Task 1: Perform single pass through array, swapping adjacent elements if `a[j] > a[j+1]`.
Task 2 & 3: Use outer loop `i` from `0` to `n-1` and inner loop `j` from `0` to `n-1-i`. Swap adjacent elements out of order. Print array.

6. PROGRAM / QUERIES / COMMANDS

Task 1: One Pass on {5, 1, 4, 2, 8}

```
public class Task1 {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int j = 0; j < a.length - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
        }
        for (int v : a) System.out.println(v);
    }
}
```

Task 2: Full Bubble Sort {5, 1, 4, 2, 8}

```
public class Task2 {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                }
            }
        }
        for (int v : a) System.out.println(v);
    }
}
```

Task 3: Bubble Sort {9, 7, 3}

```
public class Task3 {
    public static void main(String[] args) {
        int[] a = {9, 7, 3};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                }
            }
        }
        for (int v : a) System.out.println(v);
    }
}
```

7. INPUT

Arrays: `{5, 1, 4, 2, 8}` and `{9, 7, 3}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)



9. RESULT

Bubble sort was executed successfully for single pass and complete array sorting.

10. VIVA-VOCE QUESTIONS

Q1: Is Bubble Sort stable?
A: Yes, equal elements retain their relative positions.

Q2: How to optimize Bubble Sort for sorted arrays?
A: Use a `swapped` boolean flag to break early if no swaps occur in a pass.

JAVA - EXPERIMENT NO. 04
Selection Sort

1. EXPERIMENT TITLE

Implementation of Selection Sort Algorithm

2. AIM

Sort an array by finding the smallest value in the unsorted part and moving it to the front.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, IDE / Terminal

4. THEORY

Selection sort divides the list into sorted and unsorted regions. It repeatedly selects the minimum element from the unsorted region and swaps it with the leftmost unsorted element.

5. ALGORITHM / PROCEDURE

Task 1: Loop array, find index of minimum element.
Task 2: Outer loop `i` from `0` to `n-2`. Find minimum index in `[i...n-1]`. Swap `a[i]` and `a[minIdx]`.
Task 3: Perform first step only: find min index, swap with `a[0]`, print array.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Smallest Index in {64, 25, 12, 22, 11}

```
public class Task1 {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        int minIdx = 0;
        for (int i = 1; i < a.length; i++) {
            if (a[i] < a[minIdx]) minIdx = i;
        }
        System.out.println(minIdx);
    }
}
```

Task 2: Full Selection Sort

```
public class Task2 {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        for (int i = 0; i < a.length - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIdx]) minIdx = j;
            }
            int temp = a[minIdx]; a[minIdx] = a[i]; a[i] = temp;
        }
        for (int v : a) System.out.println(v);
    }
}
```

Task 3: First Step Only

```
public class Task3 {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        int minIdx = 0;
        for (int j = 1; j < a.length; j++) {
            if (a[j] < a[minIdx]) minIdx = j;
        }
        int temp = a[minIdx]; a[minIdx] = a[0]; a[0] = temp;
        for (int v : a) System.out.println(v);
    }
}
```

7. INPUT

Array: `{64, 25, 12, 22, 11}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 04

```
c:\java> java Task1 4 c:\java> java Task2 11 12 22 25 64 c:\java> java Task3 11 25 12 22 64
```

9. RESULT

Selection sort logic was successfully implemented and verified.

10. VIVA-VOCE QUESTIONS

Q1: What is the number of swaps in Selection Sort?
A: O(n) swaps, which makes it useful when memory writes are expensive.

Q2: What is the best-case time complexity of Selection Sort?
A: $O(n^2)$, since it always scans the unsorted part completely.

JAVA - EXPERIMENT NO. 05
Linear Search

1. EXPERIMENT TITLE

Sequential Linear Search Implementation

2. AIM

Search an array one value at a time and report the index where a value is found.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, Command Line / IDE

4. THEORY

Linear search checks every element sequentially until the key element is matched or array bounds are reached. Works on unsorted and sorted data with time complexity O(n).

5. ALGORITHM / PROCEDURE

Task 1 & 2: Iterate array index `i`. If `a[i] == key`, return `i`. If loop ends, return `-1`.
Task 3: Loop through array, increment counter every time element matches target value `52`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Linear Search Found (Key = 80)

```
public class Task1 {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) if (a[i] == key) return i;
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 80));
    }
}
```

Task 2: Linear Search Not Found (Key = 100)

```
public class Task2 {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) if (a[i] == key) return i;
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 100));
    }
}
```

Task 3: Count Frequency of 52

```
public class Task3 {
    public static void main(String[] args) {
        int[] a = {52, 10, 52, 7, 52};
        int count = 0;
        for (int v : a) if (v == 52) count++;
        System.out.println("Count: " + count);
    }
}
```

7. INPUT

Arrays: `{45, 67, 80, 52, 90}` and `{52, 10, 52, 7, 52}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 05

```
c:\java> java Task1 2 c:\java> java Task2 -1 c:\java> java Task3 Count: 3
```

9. RESULT

Linear search function and occurrence counter executed accurately.

10. VIVA-VOCE QUESTIONS

Q1: What is the worst-case scenario for linear search?
A: When the element is at the very last index or not present in the array at all.

Q2: Can linear search be applied to linked lists?
A: Yes, it is the primary search method for singly linked lists.

JAVA - EXPERIMENT NO. 06
Binary Search on a Sorted Array

1. EXPERIMENT TITLE

Binary Search Algorithm Implementation

2. AIM

Search a sorted array by checking the middle value and throwing away half of the array each time.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

Binary search operates on sorted arrays by repeatedly dividing search space in half. Middle element `mid` is compared with `key`. If `key < a[mid]`, search left half; else search right half. Time complexity: O(log n).

5. ALGORITHM / PROCEDURE

1. Initialize `low = 0`, `high = a.length - 1`.
2. While `low <= high`, calculate `mid = low + (high - low) / 2`.
3. If `a[mid] == key`, return `mid`. If `a[mid] < key`, set `low = mid + 1`; else set `high = mid - 1`. Return `-1` if unfound.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Search Key 40 in {10, 20, 30, 40, 50}

```
public class Task1 {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

Task 2: Search Key 35 (Absent)

```
public class Task2 {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

Task 3: Search Key 10 (First Element)

```
public class Task3 {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 10));
    }
}
```

7. INPUT

Sorted Array: `{10, 20, 30, 40, 50}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 06

```
c:\java> java Task1 3 c:\java> java Task2 -1 c:\java> java Task3 0
```

9. RESULT

Binary search algorithm successfully tested for existing, missing, and boundary elements.

10. VIVA-VOCE QUESTIONS

Q1: Why does Binary Search require a sorted array?
A: To reliably eliminate half the remaining elements at each step based on relative magnitude.

Q2: Why use `low + (high - low)/2` instead of `(low + high)/2`?
A: To prevent potential integer overflow when `low + high` exceeds `Integer.MAX_VALUE`.

JAVA - EXPERIMENT NO. 07
Practice Set

1. EXPERIMENT TITLE

Arrays, Searching, Sorting, and Loop Consolidation

2. AIM

Solve four short problems from arrays, searching, sorting and loops.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, IDE / Command Prompt

4. THEORY

Reinforcement of array searches, conditional messaging, accumulation loops, bubble sort passes, and modulo filtering (`% 2 == 0`).

5. ALGORITHM / PROCEDURE

Task 1: Loop array `{45, 52, 80}`; if 52 found print index, else "Not found".
Task 2: Sort `{3, 1, 2}` using Bubble Sort and print.
Task 3: Multiply elements of `{2, 3, 4}` using loop.
Task 4: Loop `{1, 2, 3, 4}`; print elements where `val % 2 == 0`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Conditional Search

```
public class Task1 {
    public static void main(String[] args) {
        int[] a = {45, 52, 80};
        int idx = -1;
        for (int i = 0; i < a.length; i++) if (a[i] == 52) { idx = i; break; }
        if (idx != -1) System.out.println("Found at index " + idx);
        else System.out.println("Not found");
    }
}
```

Task 2: Bubble Sort {3, 1, 2}

```
public class Task2 {
    public static void main(String[] args) {
        int[] a = {3, 1, 2};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                }
            }
        }
        for (int v : a) System.out.println(v);
    }
}
```

Task 3: Product of Array & Task 4: Filter Evens

```
public class Task3 {
    public static void main(String[] args) {
        int[] a = {2, 3, 4};
        int prod = 1;
        for (int v : a) prod *= v;
        System.out.println("Product: " + prod);
    }
}

public class Task4 {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4};
        for (int v : a) if (v % 2 == 0) System.out.println(v);
    }
}
```

7. INPUT

Arrays specified in respective task descriptions.

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 07

```
c:\java> java Task1 Found at index 1 c:\java> java Task2 1 2 3 c:\java> java Task3
Product: 24 c:\java> java Task4 2 4
```

9. RESULT

All four practice problems were correctly written and verified.

10. VIVA-VOCE QUESTIONS

Q1: What does `x % 2` return for odd numbers?
A: Returns 1 (or -1 for negative odd integers).

Q2: Why break from search loop when key is found?
A: To eliminate unnecessary iterations and optimize execution time.

JAVA - EXPERIMENT NO. 08
Classes and Objects

1. EXPERIMENT TITLE

Defining Classes, Fields, and Methods in Java

2. AIM

Write a small class with fields and a method, and create objects from it.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

A Class is a blueprint defining state (fields) and behavior (methods). An Object is an instance of a class allocated in memory. Dot operator (`.`) accesses object members.

5. ALGORITHM / PROCEDURE

Task 1 & 2: Define `Student` class with `roll`, `name`, and `show()` method. Instantiate in `main`, assign values, call `show()`.
Task 3: Define `Book` class with `pages` and `isLong()` returning `pages > 300`. Test two instances.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Single Student Object

```
class Student {
    int roll;
    String name;
    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}
public class Task1 {
    public static void main(String[] args) {
        Student s = new Student();
        s.roll = 5; s.name = "Rohan";
        s.show();
    }
}
```

Task 2: Multiple Student Objects

```
public class Task2 {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 5; s1.name = "Rohan";
        s1.show();

        Student s2 = new Student();
        s2.roll = 8; s2.name = "Meera";
        s2.show();
    }
}
```

Task 3: Book Class and Method

```
class Book {
    int pages;
    boolean isLong() { return pages > 300; }
}
public class Task3 {
    public static void main(String[] args) {
        Book b1 = new Book(); b1.pages = 450;
        Book b2 = new Book(); b2.pages = 120;
        System.out.println(b1.isLong());
        System.out.println(b2.isLong());
    }
}
```

7. INPUT

Student instances (5/Rohan, 8/Meera); Book instances (450, 120 pages).

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 08

```
      c:\java> java Task1 Roll: 5 Name: Rohan c:\java> java Task2 Roll: 5 Name: Rohan Roll:
8 Name: Meera c:\java> java Task3 true false
```

9. RESULT

Class instantiation, field assignment, and method invocation were successfully demonstrated.

10. VIVA-VOCE QUESTIONS

Q1: Where are objects stored in Java memory?
A: Objects are allocated on the Heap memory.

Q2: What is default initialization for instance variables?
A: Numeric types initialize to `0`, booleans to `false`, and references to `null`.

JAVA - EXPERIMENT NO. 09
Constructors

1. EXPERIMENT TITLE

Object Initialization using Constructors

2. AIM

Set field values when an object is created, using constructors.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

Constructors are special member functions invoked during object creation. They share the class name and have no return type. The `this` keyword resolves ambiguity between instance fields and parameter names.

5. ALGORITHM / PROCEDURE

Task 1: Define `Box` with no-arg constructor setting `width = 10`.
Task 2: Define `Box` parameterized constructor setting `this.width = width`.
Task 3: Define `Counter` constructor setting `count = 0` and `increment()` method.

6. PROGRAM / QUERIES / COMMANDS

Task 1: No-arg Constructor

```
class Box1 {
    int width;
    Box1() { width = 10; }
}
public class Task1 {
    public static void main(String[] args) {
        Box1 b = new Box1();
        System.out.println("Width: " + b.width);
    }
}
```

Task 2: Parameterized Constructor ('this' keyword)

```
class Box2 {
    int width;
    Box2(int width) { this.width = width; }
}
public class Task2 {
    public static void main(String[] args) {
        Box2 b = new Box2(25);
        System.out.println("Width: " + b.width);
    }
}
```

Task 3: State Initialization and Increment

```
class Counter {
    int count;
    Counter() { count = 0; }
    void increment() { count++; }
}
public class Task3 {
    public static void main(String[] args) {
        Counter c = new Counter();
        c.increment(); c.increment(); c.increment();
        System.out.println("Count: " + c.count);
    }
}
```

7. INPUT

Constructor arguments: 25 for Box2; 3 calls to `increment()` for Counter.

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 09

```
c:\java> java Task1 Width: 10 c:\java> java Task2 Width: 25 c:\java> java Task3 Count:
3
```

9. RESULT

Constructors were successfully implemented to set initial object state.

10. VIVA-VOCE QUESTIONS

Q1: Can a constructor be private?
A: Yes, private constructors restrict instantiation (used in Singleton design pattern).

Q2: What happens if no constructor is defined in a class?
A: Java automatically inserts a default no-argument constructor.

JAVA - EXPERIMENT NO. 10
Parent and Child Classes

1. EXPERIMENT TITLE

Single-Level Class Inheritance in Java

2. AIM

Make one class extend another, so that the child class uses the parent's fields and methods.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

Inheritance enables child classes (`subclasses`) to acquire members from parent classes (`superclasses`) using `extends`. The `super()` call invokes superclass constructors.

5. ALGORITHM / PROCEDURE

Task 1: Class `Dog` extends `Animal`. Call `sound()` from `Dog` instance.
Task 2: Class `Student` extends `Person`. Call `super(name)` in `Student` constructor.
Task 3: Class `Teacher` extends `Person`. Set `name` and invoke parent `greet()` method.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Method Inheritance

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {}

public class Task1 {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Task 2: Constructor Delegation (`super`)

```
class Person1 {
    String name;
    Person1(String name) { this.name = name; }
}
class Student1 extends Person1 {
    Student1(String name) { super(name); }
}

public class Task2 {
    public static void main(String[] args) {
        Student1 s = new Student1("Neha");
        System.out.println("Name: " + s.name);
    }
}
```

Task 3: Field and Method Inheritance

```
class Person2 {
    String name;
    void greet() { System.out.println("Hello, " + name); }
}
class Teacher extends Person2 {}

public class Task3 {
    public static void main(String[] args) {
        Teacher t = new Teacher();
        t.name = "Kiran";
        t.greet();
    }
}
```

7. INPUT

Names: "Neha" for Student1, "Kiran" for Teacher.

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 10

```
      c:\java> java Task1 Some sound c:\java> java Task2 Name: Neha c:\java> java Task3
Hello, Kiran
```

9. RESULT

Inheritance and superclass constructor propagation (`super`) were successfully executed.

10. VIVA-VOCE QUESTIONS

Q1: Does Java support multiple inheritance with classes?
A: No, to avoid diamond problem ambiguity. Interfaces are used instead.

Q2: What is the root class of all classes in Java?
A: `java.lang.Object`.

JAVA - EXPERIMENT NO. 11
Method Overriding

1. EXPERIMENT TITLE

Runtime Polymorphism via Method Overriding

2. AIM

Change a method in a child class so that it gives a different answer from the parent version.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, IDE / Terminal

4. THEORY

Method Overriding occurs when a subclass provides a specific implementation for a method declared in its parent class. Method resolution occurs at runtime (Dynamic Method Dispatch).

5. ALGORITHM / PROCEDURE

Task 1: Parent `Shape.area()` returns 0. Subclass `Square` overrides `area()` to return `side * side`.
Task 2: Subclass `Rectangle` overrides `area()` to return `length * breadth`.
Task 3: Store `Square` instance in a `Shape` reference variable and call `area()`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Square Overrides Shape

```
class Shape {
    int area() { return 0; }
}
class Square extends Shape {
    int side;
    int area() { return side * side; }
}
public class Task1 {
    public static void main(String[] args) {
        Square sq = new Square();
        sq.side = 4;
        System.out.println("Area: " + sq.area());
    }
}
```

Task 2: Rectangle Overrides Shape

```
class Rectangle extends Shape {
    int length, breadth;
    int area() { return length * breadth; }
}
public class Task2 {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 4; r.breadth = 5;
        System.out.println("Area: " + r.area());
    }
}
```

Task 3: Polymorphic Reference Variable

```
public class Task3 {
    public static void main(String[] args) {
        Square sq = new Square();
        sq.side = 3;
        Shape s = sq; // Polymorphic reference
        System.out.println("Area: " + s.area());
    }
}
```

7. INPUT

Square side: 4 (Task 1), 3 (Task 3); Rectangle: 4 x 5 (Task 2).

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 11

```
c:\java> java Task1 Area: 16 c:\java> java Task2 Area: 20 c:\java> java Task3 Area: 9
```

9. RESULT

Method overriding and dynamic dispatch were successfully verified.

10. VIVA-VOCE QUESTIONS

Q1: Can a `static` method be overridden?
A: No, static methods are bound at compile-time (method hiding).

Q2: What annotation explicitly marks overridden methods?
A: `@Override`.

JAVA - EXPERIMENT NO. 12
Recursive Factorial, Sum and Power

1. EXPERIMENT TITLE

Mathematical Problem Solving using Recursion

2. AIM

Write a method that calls itself on a smaller input until a base case is reached.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, Terminal / IDE

4. THEORY

Recursion solves problems by dividing them into identical subproblems. Every recursive function must define a base case to stop infinite calls and prevent `StackOverflowError`.

5. ALGORITHM / PROCEDURE

Task 1 Factorial: `fact(0) = 1`; `fact(n) = n * fact(n-1)`.
Task 2 Sum: `sum(0) = 0`; `sum(n) = n + sum(n-1)`.
Task 3 Power: `power(b, 0) = 1`; `power(b, e) = b * power(b, e-1)`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Recursive Factorial

```
public class Task1 {
    static int factorial(int n) {
        if (n == 0) return 1;
        return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

Task 2: Recursive Sum

```
public class Task2 {
    static int sum(int n) {
        if (n == 0) return 0;
        return n + sum(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(sum(10));
    }
}
```

Task 3: Recursive Power

```
public class Task3 {
    static int power(int base, int exp) {
        if (exp == 0) return 1;
        return base * power(base, exp - 1);
    }
    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

7. INPUT

`factorial(5)`, `sum(10)`, `power(2, 5)`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 12

```
c:\java> java Task1 120 c:\java> java Task2 55 c:\java> java Task3 32
```

9. RESULT

Recursive mathematical procedures executed correctly with valid base case terminations.

10. VIVA-VOCE QUESTIONS

Q1: What memory region tracks recursive calls?
A: The Call Stack (Stack Memory).

Q2: What is the main drawback of recursion over iteration?
A: Higher memory overhead due to call stack frames and risk of stack overflow.

JAVA - EXPERIMENT NO. 13
Recursion on Arrays

1. EXPERIMENT TITLE

Recursive Array Traversal and Computation

2. AIM

Walk through an array with an index that goes one step further in each call.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

Recursive array traversal passes an explicit index parameter `i`. Base case checks if index hits boundary (`i == a.length` or `i < 0`).

5. ALGORITHM / PROCEDURE

Task 1: `sum(a, i)`: Base `i == a.length` return 0; else return `a[i] + sum(a, i+1)`.
Task 2: `max(a, i)`: Base `i == a.length-1` return `a[i]`; else return `Math.max(a[i], max(a, i+1))`.
Task 3: `printReverse(a, i)`: Base `i < 0` return; else print `a[i]`, call `printReverse(a, i-1)`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Recursive Array Sum

```
public class Task1 {
    static int sum(int[] a, int i) {
        if (i == a.length) return 0;
        return a[i] + sum(a, i + 1);
    }
    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(sum(a, 0));
    }
}
```

Task 2: Recursive Array Maximum

```
public class Task2 {
    static int max(int[] a, int i) {
        if (i == a.length - 1) return a[i];
        return Math.max(a[i], max(a, i + 1));
    }
    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(max(a, 0));
    }
}
```

Task 3: Recursive Reverse Print

```java
public class Task3 {
    static void printReverse(int[] a, int i) {
        if (i < 0) return;
        System.out.println(a[i]);
        printReverse(a, i - 1);
    }
    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        printReverse(a, a.length - 1);
    }
}
```

7. INPUT

Array: `{3, 6, 9, 12, 15}`

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 13

```
c:\java> java Task1 45 c:\java> java Task2 15 c:\java> java Task3 15 12 9 6 3
```

9. RESULT

Recursive array operations (Sum, Max, Reverse Print) were successfully executed.

10. VIVA-VOCE QUESTIONS

Q1: How many stack frames are created for an array of size N?
A: N + 1 stack frames.

JAVA - EXPERIMENT NO. 14
Sorted Check, Search and Fibonacci

1. EXPERIMENT TITLE

Recursive Searching and Sequence Generation

2. AIM

Use recursion to check an array, find a value by index, and compute Fibonacci numbers.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, IDE / Command Prompt

4. THEORY

Recursive conditionals evaluate multi-step relational invariants (e.g. `a[i] <= a[i+1]`). Fibonacci terms are defined recursively: `F(n) = F(n-1) + F(n-2)` with base cases `F(0)=0`, `F(1)=1`.

5. ALGORITHM / PROCEDURE

Task 1 `isSorted(a, i)`: If `i == a.length-1`, return true; if `a[i] > a[i+1]`, return false; else recurse `i+1`.
Task 2 `findIndex(a, key, i)`: If `i == a.length`, return -1; if `a[i] == key`, return `i`; else recurse `i+1`.
Task 3 `fib(n)`: If `n == 0` return 0; `n == 1` return 1; else return `fib(n-1) + fib(n-2)`.

6. PROGRAM / QUERIES / COMMANDS

Task 1: Recursive Sorted Check

```
public class Task1 {
    static boolean isSorted(int[] a, int i) {
        if (i >= a.length - 1) return true;
        if (a[i] > a[i + 1]) return false;
        return isSorted(a, i + 1);
    }
    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8, 10};
        System.out.println(isSorted(a, 0));
    }
}
```

Task 2: Recursive Find Index

```
public class Task2 {
    static int findIndex(int[] a, int key, int i) {
        if (i == a.length) return -1;
        if (a[i] == key) return i;
        return findIndex(a, key, i + 1);
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(findIndex(a, 80, 0));
    }
}
```

Task 3: Recursive Fibonacci

```
public class Task3 {
    static int fib(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        System.out.println(fib(7));
    }
}
```

7. INPUT

`{2, 4, 6, 8, 10}`, `{45, 67, 80, 52, 90}` key 80, `fib(7)`.

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 14

```
c:\java> java Task1 true c:\java> java Task2 2 c:\java> java Task3 13
```

9. RESULT

Recursive array validation, search, and Fibonacci generation completed successfully.

10. VIVA-VOCE QUESTIONS

Q1: What is time complexity of naive recursive Fibonacci?
A: $O(2^n)$ due to overlapping recursive subproblems.

JAVA - EXPERIMENT NO. 15
Subsets and Arrangements

1. EXPERIMENT TITLE

Backtracking: Subsets and Permutations

2. AIM

Build answers one choice at a time, print them, and undo each choice before the next one.

3. SOFTWARE / TOOLS REQUIRED

JDK 8+, VS Code / Eclipse

4. THEORY

Backtracking explores potential solutions incrementally, undoing choices when constraints fail or paths terminate. Subsets evaluate include/exclude branches (2^N). Permutations use a `used[]` state array ($N!$).

5. ALGORITHM / PROCEDURE

Task 1: At index `i`, call without `a[i]`, then add `a[i]`, recurse, and backtrack.
Task 2 & 3: Loop through options. If `!used[i]`, mark `used[i]=true`, add character, recurse, then unmark `used[i]=false` (backtrack).

6. PROGRAM / QUERIES / COMMANDS

Task 1: Generate Subsets of {1, 2, 3}

```
public class Task1 {
    static void generateSubsets(int[] a, int idx, String current) {
        if (idx == a.length) {
            System.out.println("{" + (current.endsWith(", ") ? current.substring(0,
current.length()-2) : current) + "}");
            return;
        }
        generateSubsets(a, idx + 1, current); // Skip
        generateSubsets(a, idx + 1, current + a[idx] + ", "); // Include
    }
    public static void main(String[] args) {
        generateSubsets(new int[]{1, 2, 3}, 0, "");
    }
}
```

Task 2: Permutations of 'a', 'b', 'c' & Task 3: 'x', 'y'

```
public class Task2 {
    static void permute(char[] ch, boolean[] used, String curr) {
        if (curr.length() == ch.length) { System.out.println(curr); return; }
        for (int i = 0; i < ch.length; i++) {
            if (!used[i]) {
                used[i] = true;
                permute(ch, used, curr + ch[i]);
                used[i] = false; // backtrack
            }
        }
    }
    public static void main(String[] args) {
        permute(new char[]{'a', 'b', 'c'}, new boolean[3], "");
    }
}

public class Task3 {
    static void permute(char[] ch, boolean[] used, String curr) {
        if (curr.length() == ch.length) { System.out.println(curr); return; }
        for (int i = 0; i < ch.length; i++) {
            if (!used[i]) {
                used[i] = true;
                permute(ch, used, curr + ch[i]);
                used[i] = false; // backtrack
            }
        }
    }
    public static void main(String[] args) {
        permute(new char[]{'x', 'y'}, new boolean[2], "");
    }
}
```

7. INPUT

Set `{1, 2, 3}`, Characters `{'a','b','c'}`, and `{'x','y'}`.

8. OUTPUT (CONSOLE SCREENSHOT SIMULATION)

Console Output - Experiment 15

```
      c:\java> java Task1 {} {3} {2} {2, 3} {1} {1, 3} {1, 2} {1, 2, 3} c:\java> java Task2
abc acb bac bca cab cba c:\java> java Task3 xy yx
```

9. RESULT

Subsets and permutations were generated correctly using recursive backtracking.

10. VIVA-VOCE QUESTIONS

Q1: How many subsets are there for a set of size N?
A: 2^n subsets.

Q2: How many permutations exist for N distinct elements?
A: N! (N factorial) permutations.